

← Go Back

Hardware

Tutorials

- Debugging SAM-Based Arduino® Boards with Atmel-ICE
- Accessing the Built-in RGB LED on the MKR WiFi 1010
- How to Connect Sensors to the MKR WiFi 1010
- Connecting MKR WiFi 1010 to a Wi-Fi Network
- MKR WiFi 1010 Bluetooth® Low Energy
- Host a Web Server on the MKR WiFi 1010
- MKR WiFi 1010 Battery Application Note
- Using the Segger J-Link Debugger with the MKR Boards
- Sending Data over MQTT**
- Serial to OLED with MKR WiFi 1010
- Powering MKR WiFi 1010 with Batteries
- RTC (Real Time Clock) with MKR WiFi 1010 and OLED Display
- Scanning Networks with MKR WiFi 1010
- Securely Connecting an Arduino MKR WiFi 1010 to AWS IoT Core
- Web Server Access Point (AP) Mode with MKR WiFi 1010

Home / Hardware / MKR WiFi 1010 / **Sending Data over MQTT**

Sending Data over MQTT

Learn how to use the MQTT (Message Queuing Telemetry Transport) protocol to send data between the MKR WiFi 1010 and another device.

Author · Karl Söderby · Last revision · 07/17/2024

ON THIS PAGE

Introduction

- Goals
- Hardware & Software Needed
- Message Queuing Telemetry Transport (MQTT) +
- Step by Step +
- Testing It Out +
- Conclusion

Introduction

In this tutorial, we will create a setup that allows a MKR WiFi 1010 board to send data to another Wi-Fi compatible board, using MQTT (Message Queuing Telemetry Transport). The sender device, simply publishes a message to a broker service, which then can be subscribed to by a receiver device.

The data we will send is simply random readings from the analog inputs on the MKR WiFi 1010 board, but can easily be replaced by any sensor. This tutorial uses the broker test.mosquitto.org, an open-source service, free to use by anyone.

This tutorial uses the [ArduinoMqttClient](#) and [WiFinINA](#) libraries.

Goals

The goals of this project are:

Help

Learn some basics of how MQTT works.

Learn how to use the **ArduinoMqttClient** library.

Create a sketch for a **publisher** device.

Create a sketch for a **subscriber** device.

Hardware & Software Needed

Arduino IDE ([online](#) or [offline](#)).

[ArduinoMqttClient](#) library.

[Wi-Fi-NINA](#) library.

2x Arduino MKR WiFi 1010 ([link to store](#)).

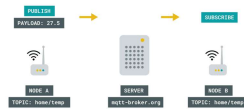
2x Micro USB cables.

Message Queuing Telemetry Transport (MQTT)

The MQTT protocol was first introduced in 1999, as a light-weight **publish** and **subscribe** system. It is particularly useful for devices with low-bandwidth, where we can send commands, sensor values or messages over the Internet with little effort.

A basic explanation on how it works is that a node, for example an Arduino with a Wi-Fi module, sends a payload to a broker. A broker is a kind of "middle-point" server, that essentially stores

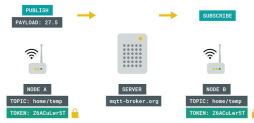
definition of what type of data it contains, it could for example be "basement humidity" or "living room temperature". Another node can then subscribe to this information, from the broker, and voilà, data has been moved from Node A to Node B over the Internet.



Basics of MQTT.

There are several different ways this can be practiced, with many different layers of security depending on what type of broker and setup we use. For example, if we are dealing with non-sensitive data, such as temperature of a specific location, we are not too concerned on who might get access to it. But there's cases where data needs to be protected, for example in Social Media messaging services.

One way to protect the data is for example, by using a **token**, something that is quite common when working with various IoT services. For instance, if we are publishing something to a broker, anyone that has the URL, e.g. **randombroker.org/randomtopic** can subscribe to it. But if we add a unique token on both sides, they wouldn't be able to. These tokens could for example be **Z6ACuLwr5T**, which is not exactly something easy to guess.



Encryption of MQTT.

MQTT is quite the interesting topic, and if you wish to read more about it, check out the links below:

mqtt.org - all you need to know about MQTT.

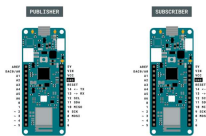
[Mosquitto.org](https://mosquitto.org) - an MQTT broker.

[Inductive automation](#) - what is MQTT?

[Randomnerdtutorials](#) - what is MQTT and how it works.

Circuit

This tutorial requires no external circuit. It does however, require two boards with a Wi-Fi module (one publisher and one subscriber).



Simple Publisher and Subscriber circuit.

Step by Step

We will now go through the steps required to setup one board as a publisher, and one as a

Include the necessary libraries.

Create a header file to store Wi-Fi credentials.

Configure the **publisher device** to create three topics and publish them to a broker.

Configure the **subscriber device** to subscribe to the three topics.

1. First, let's make sure we have the drivers installed. If we are using the Cloud Editor, we do not need to install anything. If we are using an offline editor, we need to install it manually. This can be done by navigating to **Tools > Board > Board Manager....** Here we need to look for the **Arduino SAMD boards (32-bits Arm® Cortex®-M0+)** and install it.

2. Now, we need to install the libraries needed. If we are using the Cloud Editor, there is no need to install anything. If we are using an offline editor, simply go to **Tools > Manage libraries..**, and search for **ArduinoMqttClient** and **Wi-Fi-NINA** and install them both.

3. Now let's take a look at some important functions used in the sketches:

`WiFiClient wifiClient` - creates a Wi-Fi client.

`MqttClient
mqttClient(wifiClient)` - connects the Wi-Fi client to the MQTT client.

`WiFi.begin(ssid, pass)` - connects to local Wi-Fi network.

port).

`mqttClient.poll()` -

keeps the connection alive,
used in the `loop()`.

`mqttClient.beginMessage(topic)`

- creates a new message to
be published.

`mqttClient.print()` -

prints the content of
message between the ().

`mqttClient.endMessage()`

- publishes the message to
the broker.

`mqttClient.subscribe(topic)`

- subscribes to a topic.

`mqttClient.available()` -

checks if any messages are
available from the topic.

`mqttClient.read()` - reads

the incoming messages.

Programming the Publisher

We will now program the publisher device. Let's start by opening an empty sketch, and create a header file called `arduino_secrets.h` that we can store our Wi-Fi credentials in. To create a tab in the offline editor, click the arrow symbol underneath the Serial Monitor symbol, and click on the "New tab" option.



Creating new tab.

Then, name the file



Naming the file.

Inside this new header file, we need to use the below code, where our `SECRET_SSID` (network name) and `SECRET_PASS` (password) needs to be replaced by your own credentials.

```
1 #define SECRET_SSID ""  
2 #define SECRET_PASS ""
```

We can now copy and paste the **sender** code below into our regular sketch file, and upload it to our board. Make sure we have selected the right port and board before uploading.

Note: The char `topic[]`, `topic2[]` and `topic3[]`, created here may be used by someone else. If we change this, we will also need to change the name of the topic we subscribe to in the subscriber sketch.

```
1 #include <ArduinoMqt
2 #include <WiFiNINA.h
3 #include "arduino_se
4
5 //please enter
6 char ssid[] = SECRET
7 char pass[] = SECRET
8
9 WiFiClient wifiClien
10 MqttClient mqttClien
11
12 const char broker[]
13 int port
14 const char topic[]
15 const char topic2[]
16 const char topic3[]
17
18 //set interval for s
19 const long interval
20 unsigned long previo
21
22 int count = 0;
23
24 void setup() {
25     //Initialize seria
26     Serial.begin(9600)
27     while (!Serial) {
28         ; // wait for se
```

Programming the Subscriber Device

We will now program the subscriber device. For this, we need to create a new sketch, and create another

`arduino_secrets.h` file.

We can now copy and paste the **receiver** code below into our regular sketch file, and upload it to our board. Make sure we have selected the right port and board before uploading.

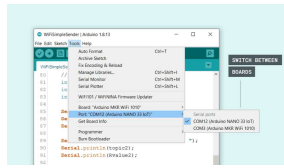

```
1  #include <ArduinoMqtt
2  #include <WiFiNINA.h>
3  #include "arduino_sec
4
5  ///////please enter y
6  char ssid[] = SECRET_
7  char pass[] = SECRET_
8
9  WiFiClient wifiClient
10  MqttClient mqttClient
11
12  const char broker[] =
13  int      port      =
14  const char topic[] =
15  const char topic2[]
16  const char topic3[]
17
18  void setup() {
19      //Initialize serial
20      Serial.begin(9600);
21      while (!Serial) {
22          ; // wait for ser
23      }
24      // attempt to conne
25      Serial.print("Attem
26      Serial.println(ssid
27      while (WiFi.begin(s
28          // failed, retry
```

Testing It Out

If everything was successful during the upload, we now have a **publisher** and **subscriber** device. Next, we need to open the Serial Monitor for each board, one at a time. This will initialize the sketch. Since we can only have one Serial Monitor open at one time, we will need to switch the ports manually. Using only one computer can be a bit tedious, as we can never view the Serial Monitor of both devices at the same time.

In this tutorial, a MKR WiFi 1010 board and a Nano 33 IoT board

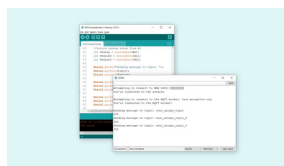
between the ports, we can see them listed as COM12 and COM3.



Selecting the board.

When both devices have been initialized, they start connecting to the network. Now, let's open the Serial Monitor and keep it open for the **publisher** device.

We can now see that we are sending messages every 8 seconds (this interval can be changed at the top of the code). Each interval sends three messages, for each topic. Each topic contains the latest reading from an analog pin.



Serial Monitor output of publisher device.

Now, let's open the **subscriber** device's Serial Monitor, and keep it open. If everything is successful, we can now see the analog values. This means, that we have successfully published three topics on one device, and subscribed to them using another device.



Serial Monitor output
of subscriber device.

Troubleshoot

If the code is not working, there are some common issues we can troubleshoot:

Check that the host is right:
this tutorial uses
test.mosquitto.org.

Check that our credentials
are correct in the
`arduino_secrets.h` tab.

Make sure that the topics we
publish match the topics we
subscribe to.

Conclusion

In this tutorial, we have created a very basic MQTT application, which allows data to flow from a publisher device, via a broker, to a subscriber device. This type of setup is commonly used in many Internet of Things (IoT) applications, and we encourage you to continue exploring the [ArduinoMqttClient](#) library. In this tutorial, in order to create a minimal working project we did not use any form of encryption. This is OK to use for just basic, non-private data, but if you are intending to use it for e.g. home automation, security systems and so on, it is **strongly recommended** to use more security layers.

The broker used in this tutorial,
test.mosquitto.org provides some

encryption, and what ports to use etc.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

